Summarize

Chapter 11. Security

I want to preface this chapter by saying that I do not consider myself to be an expert in the space of application security. I aim simply to be a *conscious incompetent*—in other words, I want to understand what I don't know and be aware of my limits. Even as I learn more about this space, I learn that there is still more to know. This isn't to say that educating yourself about topics like this is pointless—I feel that everything I've learned about this space over the last ten years has made me a more effective developer and architect.

In this chapter, I highlight those aspects of security that I consider worthwhile for a general developer, architect, or operations person working on a microservice architecture to understand. There is still a need to have the support of experts in the space of application security—but even if you have access to such people, it's still important for you to have some grounding in these topics. In the same way that developers have learned more about testing or data management, topics previously restricted to specialists, having a general awareness of security topics can be vital in terms of building security into our software from the beginning.

When comparing microservices to less distributed architectures, it turns out we are presented with an interesting dichotomy. On the one hand, we now have more data flowing over networks that previously would stay on a single machine, and we have more complex infrastructure running our architecture—our surface area of attack is much greater. On the other hand, microservices give us more opportunity to defend in depth and limit the scope of access, potentially increasing the projection of our system while also reducing the impact of an attack should it occur. This apparent paradox—that microservices can make our systems both *less* secure and *more* secure—is really just a delicate balancing act. My hope is that by the end of this chapter you'll end up on the right side of this equation.

To help you find the right balance when it comes to security of your microservice architecture, we'll cover the following topics:

Core principles

Fundamental concepts that are useful to embrace when looking to build more secure software

The five functions of cybersecurity

Identify, protect, detect, respond, and recover—an overview of the five key function areas for application security

Foundations of application security

Some specific fundamental concepts of application security and how they apply to microservices, including credentials and secrets, patching, backups, and rebuild

Implicit trust versus zero trust

Different approaches for trust in our microservice environment and how this impacts security-related activities

Securing data

How we protect data as it travels over networks and as it rests on disk

Authentication and authorization

How single sign-on (SSO) works in a microservice architecture, centralized versus decentralized authorization models, and the role of JWT tokens as part of this

Core Principles

Often when the topic of microservice security comes up, people want to start talking about reasonably sophisticated technological issues like the use of JWT tokens or the need for mutual TLS (topics we will explore later in this chapter). However, the problem with security is that you're only as secure as your least secure aspect. To use an analogy, if you're looking to secure your home, it would be a mistake to focus all your efforts on having a front door that is pick resistant, with lights and cameras to deter malicious parties, if you leave your back door open.

Thus, there are some fundamental aspects of application security that we need to look at, however briefly, to highlight the plethora of issues you need to be aware of. We will look at how these core issues are made more (or less) complex in the context of microservices, but they should also be generally applicable to software development as a whole. For those of you who want to jump ahead to all that "good stuff," please just

make sure you aren't focusing too much on securing your front door while leaving the back door open.

Principle of Least Privilege

When granting application access to individuals, external or internal systems, or even our own microservices, we should pay careful attention to what access we grant. The principle of least privilege describes the idea that when granting access we should grant the minimum access a party needs to carry out the required functionality, and only for the time period they need it. The main benefit of this is to ensure that if credentials are compromised by an attacker, those credentials will give the malicious party as limited access as possible.

If a microservice only has read-only access to a database, then an attacker gaining access to those database credentials will only gain read-only access, and only to that database. If the credential to the database expires before it is compromised, then the credential becomes useless. This concept can be extended to limit which microservices can be communicated with by specific parties.

As we'll see later in this chapter, the principle of least privilege can extend to ensuring that access controls are only granted for limited timeframes as well, further limiting any bad outcomes that can occur in the event of a compromise.

Defense in Depth

The UK, where I live, is full of castles. These castles are a partial reminder of our country's history (not least a reminder of a time before the United Kingdom was, well, united to some degree). They remind us of a time when people felt the need to defend their property from foes. Sometimes the perceived foes were different—many of the castles near where I live in Kent were designed to defend against coastal invasions from France.[1] Whatever the reason, castles can be a great example of the principle of defense in depth.

Having only one protection mechanism is a problem if an attacker finds a way to breach that defense, or if the protection mechanism defends against only certain types of attackers. Think of a coastal defense fort whose only wall faces the sea, leaving it totally defenseless against an attack by land. If you look at Dover Castle, which is very close to where I live, there are multiple protections that are easy to see. Firstly, it is on a large hill, making an approach to the castle by land difficult. It has not one wall but two—a

breach in the first wall still requires that an attacker deals with the second one. And once you're through the final wall, you have a large and imposing keep (tower) to deal with.

The same principle must apply when we build protections into our application security. Having multiple protections in place to defend against attackers is vital. With microservice architectures, we have many more places where we are able to protect our systems. By breaking our functionality apart into different microservices and limiting the scope of what those microservices can do, we're already applying defense in depth. We can also run microservices on different network segments, apply network-based protections at more places, and even make use of a mix of technology for building and running these microservices such that a single zero-day exploit may not impact everything we have.

Microservices provide more ability to defend in depth than equivalent single-process monolithic applications, and as a result, they can help organizations build more secure systems.

TYPES OF SECURITY CONTROLS

When considering the security controls we might put in place to secure our system, we can categorize them as:[2]

Preventative

Stopping an attack from happening. This includes storing secrets securely, encrypting data at rest and in transit, and implementing proper authentication and authorization mechanisms.

Detective

Alerting you to the fact that an attack is happening/has happened. Application firewalls and intrusion detection services are good examples.

Responsive

Helping you respond during/after an attack. Having an automated mechanism to rebuild your system, working backups to recover data, and a proper comms plan in place in the wake of an incident can be vital.

A combination of all three will be required to properly secure a system, and you may have multiples of each type. Coming back to our castle example, we might have multiple walls, which represent multiple preventative controls. We could have watchtowers in place and a beacon system so we can see if an attack is happening. Finally, we might have some carpenters and stonemasons on standby in case we need to strengthen the doors or walls in the wake of an attack. Obviously, you are unlikely to be building castles for a living, so we'll look at examples of these controls for a microservice architecture later in this chapter.

Automation

A recurring theme of this book is automation. As we have so many more moving parts with microservice architectures, automation becomes key to helping us manage the growing complexity of our system. At the same time, we have a drive toward increasing the speed of delivery, and automation is essential here. Computers are way better at doing the same thing over and over again than humans are—they do it faster and more efficiently than we can (and with less variability as well). They can also reduce human error and make it easier to implement the principle of least privilege—we can assign specific privileges to specific scripts, for example.

As we'll see throughout this chapter, automation can help us recover in the wake of an incident. We can use it to revoke and rotate security keys and also make use of tooling to help detect potential security issues more easily. As with other aspects of microservice architecture, embracing a culture of automation will help you immensely when it comes to security.

Build Security into the Delivery Process

Like so many other aspects of software delivery, security is all too often considered an afterthought. Historically at least, addressing security aspects of a system is something that is done after the code has been written, potentially leading to significant reworking later on. Security has often been looked at as something of a blocker to getting software out the door.

Over the last 20 years, we've seen similar issues with testing, usability, and operations. These aspects of software delivery were frequently delivered in siloed fashion, often after the bulk of the code was finished. An old colleague of mine, Jonny Schneider, once likened the approach toward usability of software in terms of a "Do you want fries with that?" mentality. In other words, usability is an afterthought—something you sprinkle on top of the "main meal."[3]

The reality, of course, is that software that isn't usable, isn't secure, can't operate properly in production, and is riddled with bugs isn't in any way shape or form a "main meal"—it's a flawed offering at best. We've gotten better at pulling testing forward into the main delivery process, as we've done with operational aspects (DevOps, anyone?) and usability—security should be no different. We need to ensure that developers have a more general awareness of security-related concerns, that specialists find a way to embed themselves into delivery teams when required, and that tooling improves to allow us to build security-related thinking into our software.

This can create challenges in organizations adopting stream-aligned teams with increased degrees of autonomy around the ownership of their microservices. What is the role of security experts? In "Enabling Teams", we'll look at how specialists like security experts can work to support stream-aligned teams and help microservice owners build more security thinking into their software, but also to make sure that you have the right deep expertise on hand when you need it.

There are automated tools that can probe our systems for vulnerabilities, such as by looking for cross-site scripting attacks. The Zed Attack Proxy (aka ZAP) is a good example. Informed by the work of OWASP, ZAP attempts to re-create malicious attacks on

your website. Other tools exist that use static analysis to look for common coding mistakes that can open up security holes, such as Brakeman for Ruby; there are also tools like Snyk, which among other things can pick up dependencies on third-party libraries that have known vulnerabilities. Where these tools can be easily integrated into normal CI builds, integrating them into your standard check-ins is a great place to start. Of course, it's worth noting that many of these types of tools can address only local issues—for example, a vulnerability in a specific piece of code. They don't replace the need to understand the security of your system at a wider, systemic level.

The Five Functions of Cybersecurity

With those core principles at the back of our minds, let's now consider the broad-ranging security-related activities that we need to perform. We'll then go on to understand how these activities vary in the context of a microservice architecture. The model I prefer for describing the universe of application security comes from the US National Institute of Standards and Technology (NIST), which outlines a useful five-part model for the various activities involved in cybersecurity:

- *Identify* who your potential attackers are, what targets they are trying to acquire, and where you are most vulnerable.
- *Protect* your key assets from would-be hackers.
- *Detect* if an attack has happened, despite your best efforts.
- *Respond* when you've found out something bad has occurred.
- *Recover* in the wake of an incident.

I find this model to be especially useful due to its holistic nature. It's very easy to put all your effort into protecting your application without first considering what threats you might actually face, let alone working out what you might do if a clever attacker does slip past your defenses.

Let's explore each of these functions in a bit more depth and look at how a microservice architecture might change how you approach these ideas in comparison to a more traditional monolithic architecture.

Identify

Before we can work out what we should protect, we need to work out who might be after our stuff and what exactly they might be looking for. It's often hard to put ourselves into the mindset of an attacker, but that's exactly what we need to do to ensure that we

focus our efforts in the right place. Threat modeling is the first thing you should look at when addressing this aspect of application security.

As human beings, we are quite bad at understanding risk. We often fixate on the wrong things while ignoring the bigger problems that can be just out of sight. This of course extends into the field of security. Our understanding about what security risks we might be exposed to is often largely colored by our limited view of the system, our skills, and our experiences.

When I chat with developers about security risks in the context of microservice architecture, they immediately start talking about JWTs and mutual TLS. They reach for technical solutions to technical problems they have some visibility of. I don't mean to point the finger just at developers—all of us have a limited view of the world. Coming back to the analogy we used earlier, this is how we can end up with an incredibly secure front door and a wide-open back door.

At one company I worked at, there had been a number of discussions around the need to install closed-circuit TV (CCTV) cameras in the reception areas of the company's offices all over the world. This was due to an incident in which an unauthorized person had gained access to the front office area and then the corporate network. The belief was that a CCTV camera system would not only deter others from trying the same thing again but also help identify the individuals involved after the fact.

The specter of corporate surveillance unleashed a wave of anguish in the company regarding "big brother"–type concerns. Depending on which side of the argument you were on, this was either about spying on employees (if you were "pro" cameras) or about being happy for intruders to gain access to the buildings (if you were "anti" cameras). Putting aside the problematic nature of such a polarized discussion,[4] one employee spoke up in a rather sheepish manner to suggest that perhaps the discussion was a bit misguided as we were missing some bigger issues—namely the fact that people seemed unconcerned that the front door of one of the main offices had a faulty lock, and that for years people would arrive in the morning to find the door unlocked.

This extreme (but true) story is a great example of a common problem we face when trying to secure our systems. Without having time to take in all the factors and understand just where your biggest risks are, you may well end up missing the places where your time is better spent. The goal of threat modeling is about helping you understand what an attacker might want from your system. What are they after? Will different types of malicious actors want to gain access to different assets? Threat modeling, when done right, is largely about putting yourself in the mind of the attacker, thinking from the out-

side in. This outsider view is important, and it's one reason why having an external party help drive a threat modeling exercise can be very useful.

The core idea of threat modeling doesn't change much when we look at microservice architectures, aside from the fact that any architecture being analyzed may now be more complex. What shifts is how we take the outcome of a threat model and put it into action. One of the outputs of a threat modeling exercise would be a list of recommendations for what security controls need to be put in place—these controls can include things like a change in process, a shift in technology, or perhaps a modification in system architecture. Some of these changes may be cross-cutting and could impact multiple teams and their associated microservices. Others might result in more targeted work. Fundamentally, though, when undertaking threat modeling, you do need to look holistically—focusing this analysis on too small a subset of your system, such as one or two microservices, may result in a false sense of security. You may end up focusing your time on building a fantastically secure front door, only to have left the window open.

For a deeper dive into this topic, I can recommend *Threat Modeling: Designing for Security*[5] by Adam Shostack.

Protect

Once we've identified our most valuable—and most vulnerable—assets, we need to ensure that they are properly protected. As I've noted, microservice architectures arguably give us a much wider surface area of attack, and we therefore have more things that may need to be protected, but they also give us more options to defend in depth. We'll spend most of this chapter focusing on various aspects of protection, primarily because this is the area where microservice architectures create the most challenges.

Detect

With a microservice architecture, detecting an incident can be more complex. We have more networks to monitor and more machines to keep an eye on. The sources of information are greatly increased, which can make detecting problems all the more difficult. Many of the techniques we explored in Chapter 10, such as log aggregation, can help us gather information that will help us detect that something bad might be happening. In addition to those, there are special tools like intrusion detection systems that you might run to spot bad behavior. The software to deal with the increasing complexity of our systems is improving, especially in the space of container workloads with tools like Aqua.

Respond

If the worst has happened and you've found out about it, what should you do? Working out an effective incident response approach is vital to limiting the damage caused by a breach. This normally starts with understanding the scope of the breach and what data has been exposed. If the data exposed includes personally identifiable information (PII), then you need to follow both security and privacy incident response and notification processes. This may well mean that you have to talk to different parts of your organization, and in some situations you may be legally obliged to inform an appointed data protection officer when certain types of breach occur.

Many an organization has compounded the impact of a breach through its mishandling of the aftermath, often leading to increased financial penalties quite aside from the damage caused to its brand and its relationship with customers. It's important, therefore, to understand not just what you have to do because of legal or compliance reasons but also what you should do in terms of looking after the users of your software. The GDPR, for example, requires that personal data breaches are reported to the relevant authorities within 72 hours—a timeline that doesn't seem overly onerous. That doesn't mean you couldn't strive to let people know earlier if a breach of their data has occurred.

Aside from the external communication aspects of response, how you handle things internally is also critical. Organizations that have a culture of blame and fear are likely going to fare badly in the wake of a major incident. Lessons won't be learned, and contributing factors won't come to light. On the other hand, an organization that focuses on openness and safety will be best placed to learn the lessons that ensure that similar incidents are less likely to happen. We'll come back to this in "Blame".

Recover

Recovery refers to our ability to get the system up and running again in the wake of an attack, and also our ability to implement what we have learned to ensure problems are less likely to happen again. With a microservice architecture, we have many more moving parts, which can make recovery more complex if an issue has a broad impact. So later in this chapter we will look at how simple things like automation and backups can help you rebuild a microservice system on demand and get your system back up and running as quickly as possible.

Foundations of Application Security

OK, now that we have some core principles in place and some sense of the wide world that security activities can cover, let's look at a few foundational security topics in the context of a microservice architecture if you want to build a more secure system—credentials, patching, backups, and rebuild.

Credentials

Broadly speaking, credentials give a person (or computer) access to some form of restricted resource. This could be a database, a computer, a user account, or something else. With a microservice architecture, in terms of how it compares to an equivalent monolithic architecture, we likely have the same number of humans involved, but we have *lots* more credentials in the mix representing different microservices, (virtual) machines, databases, and the like. This can lead to a degree of confusion around how to restrict (or not restrict) access, and in many cases it can lead to a "lazy" approach in which a small number of credentials with broad privileges are used in an attempt to simplify things. This can in turn lead to more problems if credentials are compromised.

We can break the topic of credentials down into two key areas. Firstly, we have the credentials of the users (and operators) of our system. These are often the weakest point of our system and are commonly used as an attack vector by malicious parties, as we'll see in a moment. Secondly, we can consider secrets—pieces of information that are critical to running our microservices. Across both sets of credentials, we have to consider the issues of rotation, revocation, and limiting scope.

User credentials

User credentials, such as email and password combinations, remain essential to how many of us work with our software, but they also are a potential weak spot when it comes to our systems being accessed by malicious parties. Verizon's 2020 Data Breach Investigations Report found that some form of credential theft was used in 80% of cases caused by hacking. This includes situations in which credentials were stolen through mechanisms like phishing attacks or where passwords were brute-forced.

There is some excellent advice out there about how to properly handle things like passwords—advice that despite being simple and clear to follow is still not being adopted widely enough. Troy Hunt has an excellent overview of the latest advice from both NIST and the UK's National Cyber Security Centre.[6] This advice includes recommendations to use password managers and long passwords, to avoid the use of complex password

rules, and—somewhat surprisingly—to avoid mandated regular password changes. Troy's full post is worth reading in detail.

In the current era of API-driven systems, our credentials also extend to managing things like API keys for third-party systems, such as accounts for your public cloud provider. If a malicious party gains access to your root AWS account, for example, they could decide to destroy everything running in that account. In one extreme example, such an attack resulted in a company called Code Spaces[7] going out of business—all of their resources were running in a single account, backups and all. The irony of Code Spaces offering "Rock Solid, Secure and Affordable Svn Hosting, Git Hosting and Project Management" is not lost on me.

Even if someone gets hold of your API keys for your cloud provider and doesn't decide to destroy everything you've built, they might decide to spin up some expensive virtual machines to run some bitcoin mining in the hope that you won't notice. This happened to one of my clients, who discovered that someone had spent over $10K doing just this before the account was shut down. It turns out that attackers know how to automate too—there are bots out there that will just scan for credentials and try to use them to launch machines for cryptocurrency mining.

Secrets

Broadly speaking, secrets are critical pieces of information that a microservice needs to operate and that are also sensitive enough that they require protecting from malicious parties. Examples of secrets that a microservice might need include:

- Certificates for TLS
- SSH keys
- Public/private API keypairs
- Credentials for accessing databases

If we consider the life cycle of a secret, we can start to tease apart the various aspects of secrets management that might require different security needs:

Creation

How do we create the secret in the first place?

Distribution

Once the secret is created, how do we make sure it gets to the right place (and only the right place)?

Storage

: Is the secret stored in a way that ensures only authorized parties can access it?

Monitoring

: Do we know how this secret is being used?

Rotation

: Are we able to change the secret without causing problems?

If we have a number of microservices, each of which might want different sets of secrets, we're going to need to use tooling to help manage all of this.

Kubernetes provides a built-in secrets solution. It is somewhat limited in terms of functionality but does come as part of a basic Kubernetes install, so it could be good enough for many use cases.[8]

If you were looking for a more sophisticated tool in this space, Hashicorp's Vault is worth a look. An open source tool with commercial options available, it's a veritable Swiss Army knife of secrets management, handling everything from the basic aspects of distributing secrets to generating time-limited credentials for databases and cloud platforms. Vault has the added benefit that the supporting consul-template tool is able to dynamically update secrets in a normal configuration file. This means parts of your system that want to read secrets from a local filesystem don't need to change to support the secrets management tool. When a secret is changed in Vault, consul-template can update this entry in the configuration file, allowing your microservices to dynamically change the secrets they are using. This is fantastic for managing credentials at scale.

Some public cloud providers also offer solutions in this space; for example, AWS Secrets Manager or Azure's Key Vault come to mind. Some people dislike the idea of storing critical secret information in a public cloud service like this, however. Again, this comes down to your threat model. If it is a serious concern, there is nothing to stop you from running Vault on your public cloud provider of choice and handling that system yourself. Even if data at rest is stored on the cloud provider, with the appropriate storage backend you can ensure that the data is encrypted in such a way that even if an outside party got hold of the data they couldn't do anything with it.

Rotation

Ideally, we want to rotate credentials frequently to limit the damage someone can do if they gain access to the credentials. If a malicious party gains access to your AWS API public/private keypair but that credential is changed once a week, they have only one week in which to make use of the credentials. They can still do a lot of damage in a week of course, but you get the idea. Some types of attackers like to gain access to systems and then stay undetected, allowing them to collect more valuable data over time and find ways into other parts of your system. If they used stolen credentials to gain access, you may be able to stop them in their tracks if the credentials they use expire before they can make much use of them.

A great example of rotation for operator credentials would be the generation of time-limited API keys for using AWS. Many organizations now generate API keys on the fly for their staff, with the public and private keypair being valid only for a short period of time —typically less than an hour. This allows you to generate the API keys you need to carry out whatever operation is needed, safe in the knowledge that even if a malicious party subsequently gains access to these keys, they will be unable to make use of them. Even if you did accidentally check that keypair into public GitHub, it would be no use to anyone once it's expired.

The use of time-limited credentials can be useful for systems too. Hashicorp's Vault can generate time-limited credentials for databases. Rather than your microservice instance reading database connection details from a configuration store or a text file, they can instead be generated on the fly for a specific instance of your microservice.

Moving to a process of frequent rotation of credentials like keys can be painful. I've spoken to companies that have experienced incidents as a result of key rotation, where systems stop working when keys are changed. This is often due to the fact that it can be unclear as to what is using a particular credential. If the scope of the credential is limited, the potential impact of rotation is significantly reduced. But if the credential has broad use, working out the impact of a change can be difficult. This isn't to put you off rotation but just to make you aware of the potential risks, and I remain convinced it is the right thing to do. The most sensible way forward would likely be to adopt tooling to help automate this process while also limiting the scope of each set of credentials at the same time.

Revocation

Having a policy in place to ensure that key credentials are rotated on a regular basis can be a sensible way to limit the impact of credential leakage, but what happens if you *know* that a given credential has fallen into the wrong hands? Do you have to wait until a scheduled rotation kicks in for that credential to no longer be valid? That may not be practical—or sensible. Instead, you'd ideally like to be able to automatically revoke and potentially regenerate credentials when something like this happens.

The use of tools that allow for centralized secrets management can help here, but this may require that your microservices are able to reread newly generated values. If your microservice is directly reading secrets from something like the Kubernetes secrets store or Vault, it can be notified when these values have changed, allowing your microservice to make use of the changed values. Alternatively, if your microservice reads these secrets only on startup, then you may need to do a rolling restart of your system to reload these credentials. If you are regularly rotating credentials, though, chances are you've already had to solve the problem of your microservices being able to reread this information. If you're comfortable with regular rotation of credentials, chances are you're already set up to handle emergency revocation as well.

SCANNING FOR KEYS

Accidentally checking in private keys to source code repositories is a common way for credentials to be leaked to unauthorized parties—it happens a surprising amount. GitHub automatically scans repositories for some types of secrets, but you can also run your own scanning too. It would be great if you could pick up secrets before you check in, and git-secrets lets you do just that. It can scan existing commits for potential secrets, but by setting it up as a commit hook, it can stop commits even being made. There is also the similar gitleaks, which, in addition to supporting pre-commit hooks and general scanning of commits, has a few features that make it potentially more useful as a general tool for scanning local files.

Limiting scope

Limiting the scope of credentials is core to the idea of embracing the principle of least privilege. This can apply across all forms of credentials, but limiting the scope of what a given set of credentials gives you access to can be incredibly useful. For example, in Figure 11-1, each instance of the `Inventory` microservice is given the same username and password for the supporting database. We also provide read-only access to the sup-

porting Debezium process that will be used to read data and send it out via Kafka as part of an existing ETL process. If the username and password for the microservices are compromised, an external party could theoretically gain read and write access to the database. If they gained access to the Debezium credentials, though, they'd only have read-only access.

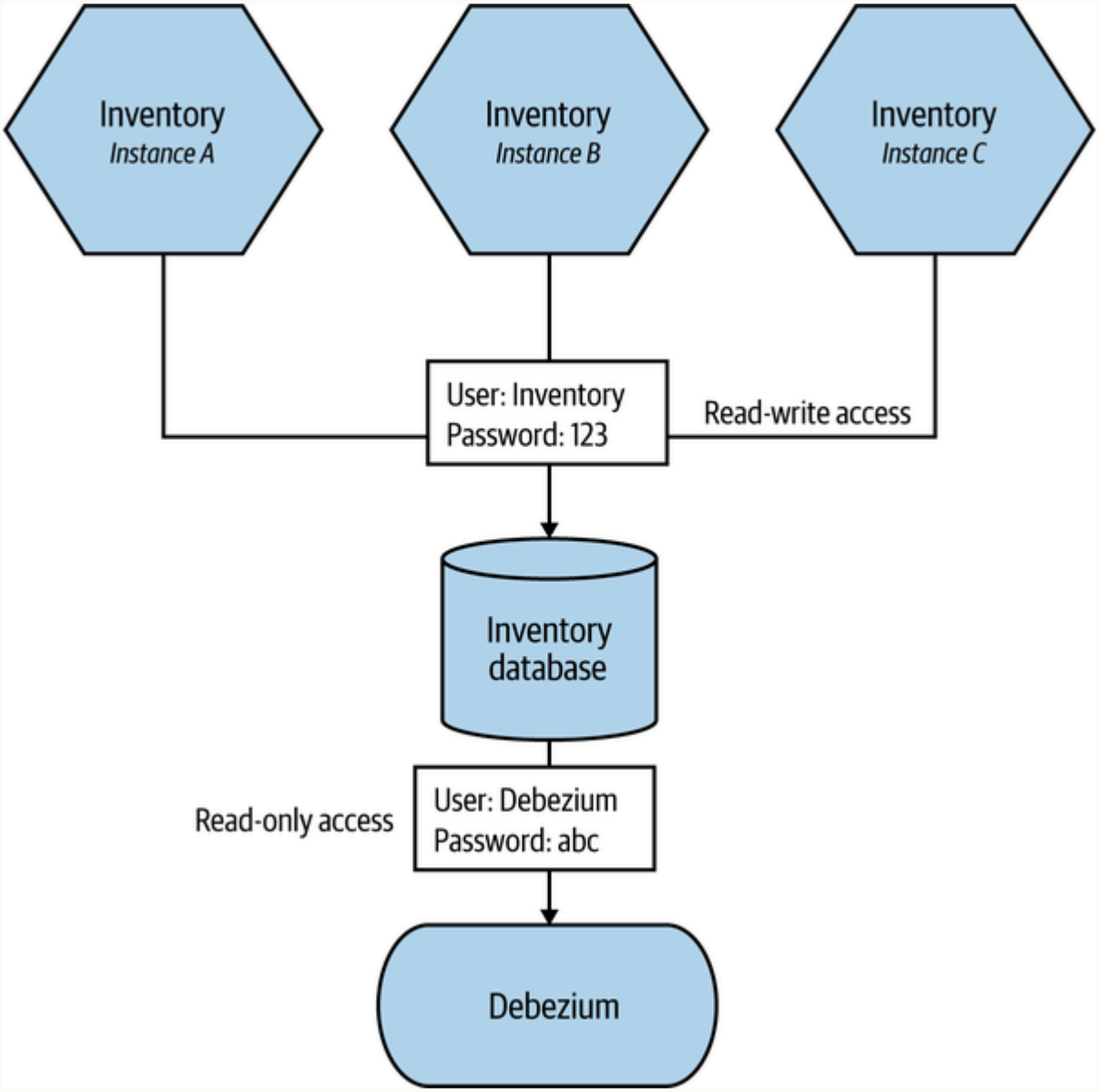


Figure 11-1. Limiting scope of credentials to limit the impact of misuse

Limiting scope can apply in terms of both what the set of credentials can access and also who has access to that set of credentials. In Figure 11-2, we've changed things such that each instance of `Inventory` gets a different set of credentials. This means that we could rotate each credential independently, or just revoke the credential for one of the instances if that is what becomes compromised. Moreover, with more specific credentials it can be easier to find out from where and how the credential was obtained. There are obviously other benefits that come from having a uniquely identifiable username for a microservice instance here; it might be easier to track down which instance caused an expensive query, for example.

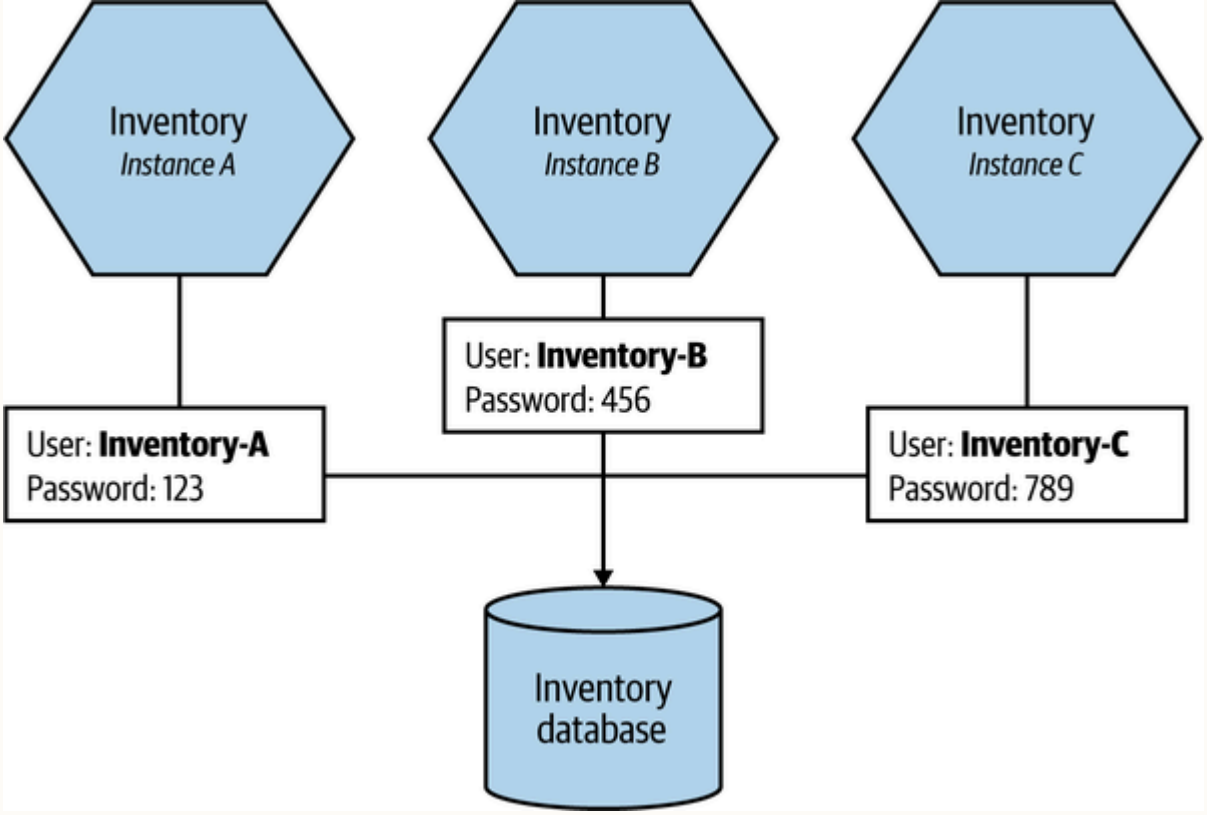


Figure 11-2. Each instance of `Inventory` has its own access credentials for the database, further limiting access

As we've already covered, managing fine-grained credentials at scale can be complex, and if you did want to adopt an approach like this, some form of automation would be essential—secret stores like Vault come to mind as perfect ways to implement schemes like this.

Patching

The 2017 Equifax data breach is a great example of the importance of patching. A known vulnerability in Apache Struts was used to gain unauthorized access to data held by Equifax. Because Equifax is a credit bureau, this information was especially sensitive. In the end it was found that the data of more than 160 million people was compromised in the breach. Equifax ended up having to pay a $700 million settlement.

Months before the breach, the vulnerability in Apache Struts had been identified, and a new release had been made by the maintainers fixing the issue. Unfortunately, Equifax hadn't updated to the new version of the software, despite it being available for months before the attack. Had Equifax updated this software in a timely manner, it seems probable that the attack wouldn't have been possible.

The issue of keeping on top of patching is becoming more complex as we deploy increasingly complex systems. We need to become more sophisticated in how we handle this fairly basic concept.

Figure 11-3 shows an example of the layers of infrastructure and software that exist underneath a typical Kubernetes cluster. If you run all of that infrastructure yourself, you're in charge of managing and patching all those layers. How confident are you that you are up to date with your patching? Obviously, if you can offload some of this work to a public cloud provider, you can offload part of this burden too.

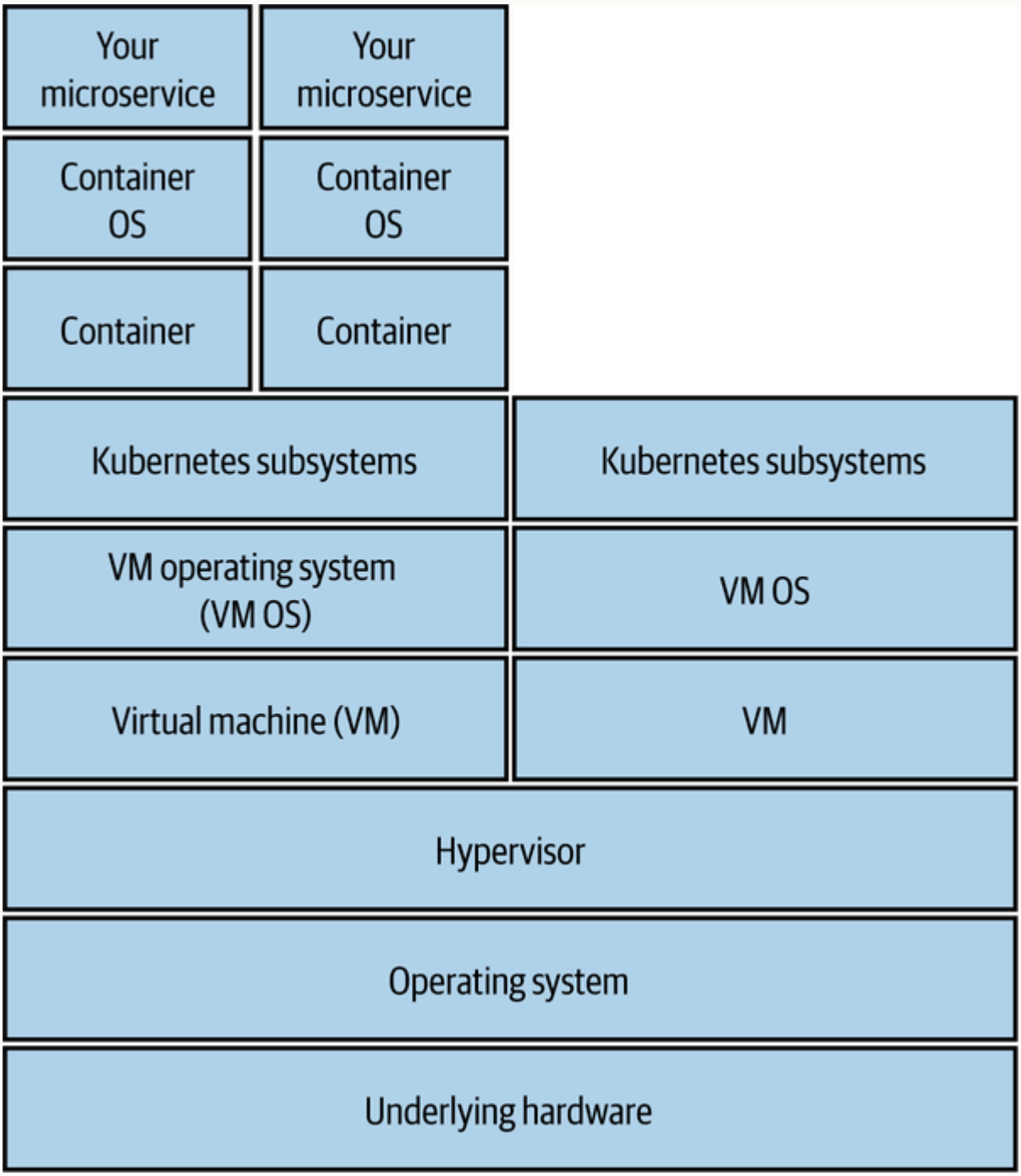


Figure 11-3. The different layers in a modern infrastructure that all require maintenance and patching

If you were to use a managed Kubernetes cluster on one of the main public cloud vendors, for example, you'd drastically reduce your scope of ownership, as we see in Figure 11-4.

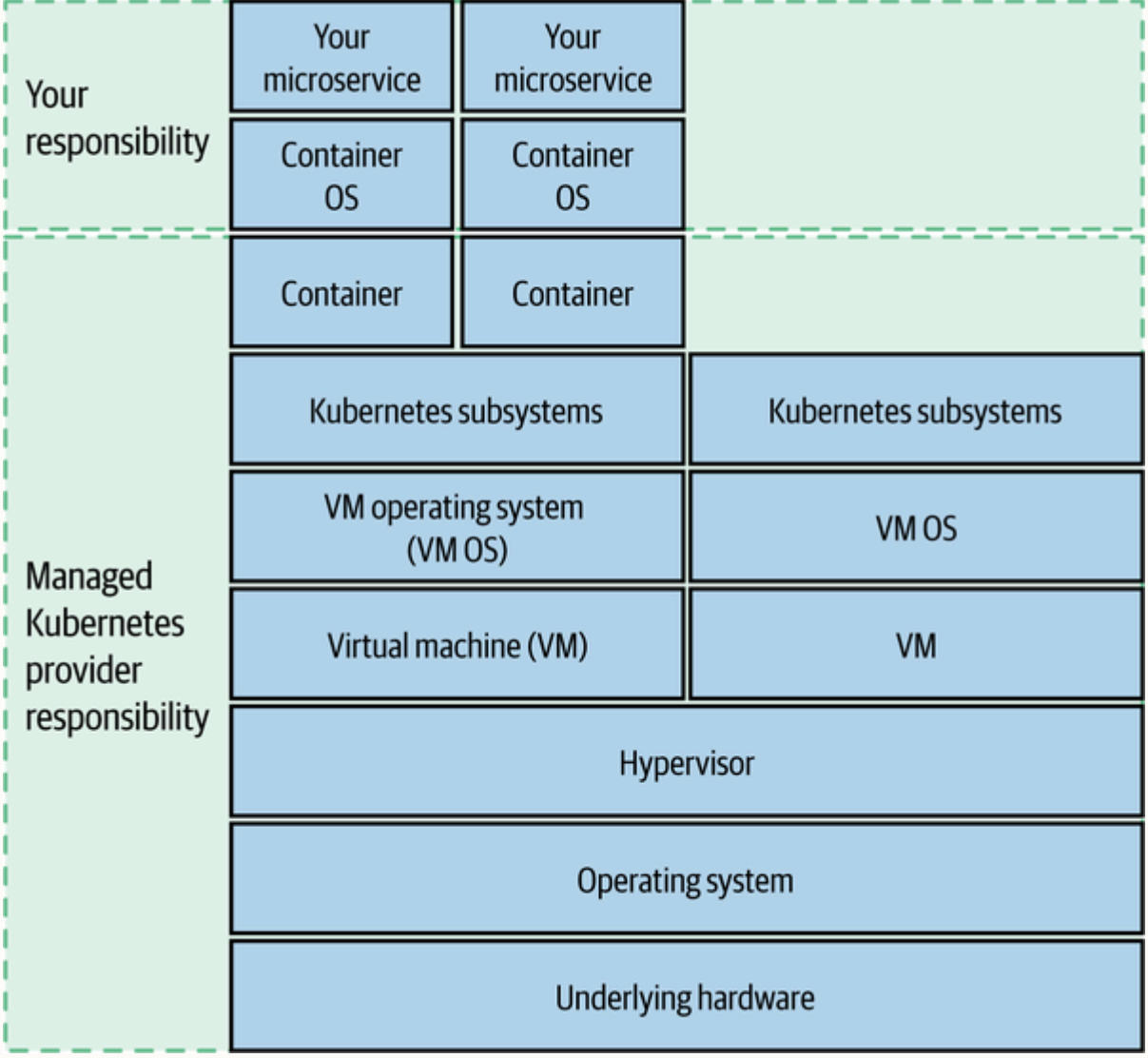


Figure 11-4. Offloading responsibility for some layers of this stack can reduce complexity

Containers throw us an interesting curveball here. We treat a given container instance as immutable. But a container contains not just our software but also an operating system. And do you know where that container has come from? Containers are based on an image, which in turn can extend other images—are you sure the base images you are using don't already have backdoors in them? If you haven't changed a container instance in six months, that's six months worth of operating system patches that haven't been applied. Keeping on top of this is problematic, which is why companies like Aqua provide tooling to help you analyze your running production containers so you can understand what issues need to be addressed.

At the very top of this set of layers, of course, is our application code. Is that up to date? It's not just the code *we* write; what about the third-party code we use? A bug in a third-party library can leave our application vulnerable to attack. In the case of the Equifax breach, the unpatched vulnerability was actually in Struts—a Java web framework.

At scale, working out which microservices are linking to libraries with known vulnerabilities can be incredibly difficult. This is an area in which I strongly recommend the use of tools like Snyk or GitHub code scanning, which is able to automatically scan your third-

party dependencies and alert you if you are linking to libraries with known vulnerabilities. If it finds any, it can send you a pull request to help update to the latest patched versions. You can even build this into your CI process and have a microservice build fail if it links to libraries with issues.

Backups

So I sometimes think that taking backups is like flossing, in that lots more people say they do it than actually do. I don't feel the need to restate the argument for backups too much here, other than to say: you should take backups, because data is valuable, and you don't want to lose it.

Data is more valuable than ever, and yet I sometimes wonder if improvements in technology have caused us to deprioritize backups. Disks are more reliable than they used to be. Databases are more likely to have built-in replication to avoid data loss. With such systems, we may convince ourselves that we don't need backups. But what if a catastrophic error occurs and your entire Cassandra cluster is wiped out? Or what if a coding bug means your application actually deletes valuable data? Backups are as important as ever. So please, back up your critical data.

With the deployment of our microservices being automated, we don't need to take full machine backups, as we can rebuild our infrastructure from source code. So we're not trying to copy the state of entire machines; we instead target our backups to the state that is most valuable. This means that our focus for backups is limited to things like the data in our databases, or perhaps our application logs. With the right filesystem technology, it's possible to take near-instantaneous block-level clones of a database's data without noticeably interrupting service.

AVOID THE SCHRÖDINGER BACKUP

When creating backups, you want to avoid what I call the Schrödinger backup.[9] This is a backup that may or may not actually be a backup. Until you actually try and restore it, you really don't know if it's actually a backup use,[10] or if it's just a bunch of 1s and 0s written to disk. The best way to avoid this problem is to ensure that the backup is real by actually restoring it. Find ways to build regular restoration of backups into your software development process—for example, by using production backups to build your performance test data.

The "old" guidance about backups is that they should be kept off-site, the idea being that an incident in your offices or data center wouldn't affect your backups if they were

elsewhere. What does "off-site" mean, though, if your application is deployed in a public cloud? What matters is that your backups are stored in a way that they are as isolated as possible from your core systems, so that a compromise in the core system won't also put your backups at risk. Code Spaces, which we mentioned earlier, had backups—but they were stored on AWS in the same account that was compromised. If your application runs on AWS, you could still store your backups there too, but you should do so on a separate account on separate cloud resources—you might even want to consider putting them into a different cloud region to mitigate the risk of a region-wide issue, or you could even store them with another provider.

So make sure you back up critical data, keep those backups in a system separate to your main production environment, and make sure the backups actually work by regularly restoring them.

Rebuild

We can do our best to ensure that a malicious party doesn't gain access to our systems, but what happens if they do? Well, often the most important thing you can do in the initial aftermath is get the system up and running again, but in such a way that you have removed access from the unauthorized party. This isn't always straightforward, though. I recall one of our machines being hacked many years ago by a rootkit. A rootkit is a bundle of software that is designed to hide the activities of an unauthorized party, and it's a technique commonly used by attackers who want to remain undetected, allowing them time to explore the system. In our case, we found that the rootkit had altered core system commands like `ls` (listing files) or `ps` (to show process listings) to hide traces of the external attacker. We spotted this only when we were able to check the hashes of the programs running on the machine against the official packages. In the end, we basically had to reinstall the entire server from scratch.

The ability to simply wipe a server from existence and totally rebuild can be incredibly effective not just in the wake of a known attack but also in terms of reducing the impact of persistent attackers. You might not be aware of the presence of a malicious party on your system, but if you are routinely rebuilding your servers and rotating credentials, you may be drastically limiting the impact of what they can do without your realizing it.

Your ability to rebuild a given microservice, or even an entire system, comes down to the quality of your automation and backups. If you can deploy and configure each microservice from scratch based on information stored in source control, you're off to a good start. Of course, you need to combine that with a rock-solid backup restoration process for the data. As with backups, the best way to make sure automated deployment and

configuration of your microservices works is to do it a lot—and the easiest way to achieve that is just to use the same process for rebuilding a microservice as you do for every deployment. This is of course how most container-based deployment processes work. You deploy a new set of containers running the new version of your microservice and shut down the old set. Making this normal operating procedure makes a rebuild almost a nonevent.

There is one caveat here, especially if you are deploying on a container platform like Kubernetes. You might be frequently blowing away and redeploying container instances, but what about the underlying container platform itself? Do you have the ability to rebuild that from scratch? If you're using a fully managed Kubernetes provider, spinning up a new cluster might not be too difficult, but if you've installed and manage the cluster yourself, then this may be a nontrivial amount of work.

TIP

Being able to rebuild your microservice and recover its data in an automated fashion helps you recover in the wake of an attack and also has the advantage of making your deployments easier across the board, having positive benefits for development, test, and production operations activities.

Implicit Trust Versus Zero Trust

Our microservice architecture consists of lots of communication between things. Human users interact with our system via user interfaces. These user interfaces in turn make calls to microservices, and microservices end up calling yet more microservices. When it comes to application security, we need to consider the issue of trust among all those points of contact. How do we establish an acceptable level of trust? We'll explore this topic shortly in terms of authentication and authorization of both humans and microservices, but before that we should consider some fundamental models around trust.

Do we trust everything running in our network? Or do we view everything with suspicion? Here, we can consider two mindsets—implicit trust and zero trust.

Implicit Trust

Our first option could be to just assume that any calls to a service made from inside our perimeter are implicitly trusted.

Depending on the sensitivity of the data, this might be fine. Some organizations attempt to ensure security at the perimeter of their networks, and they therefore assume they don't need to do anything else when two services are talking together. However, should an attacker penetrate your network, all hell could break loose. If the attacker decides to intercept and read the data being sent, change the data without you knowing, or even in some circumstances pretend to be the thing you are talking to, you may not know much about it.

This is by far the most common form of inside-perimeter trust I see in organizations. I'm not saying that is a good thing! For most of the organizations I see using this model, I worry that the implicit trust model is not a conscious decision; rather, people are unaware of the risks in the first place.

Zero Trust

> *"Jill, we've traced the call—it's coming from inside the house!"*
>
> —*When a Stranger Calls*

When operating in a zero-trust environment, you have to *assume* that you are operating in an environment that has already been compromised—the computers you are talking to could have been compromised, the inbound connections could be from hostile parties, the data you are writing could be read by bad people. Paranoid? Yes! Welcome to zero trust.

Zero trust, fundamentally, is a mindset. It's not something you can magically implement using a product or tool; it's an idea and that idea is that if you operate under the assumption that you are operating in a hostile environment in which bad actors could already be present, then you have to carry out precautions to make sure that you can still operate safely. In effect, the concept of a "perimeter" is meaningless with zero trust (for this reason, zero trust is often also known as "perimeterless computing").

Since you are assuming your system has been compromised, all inbound calls from other microservices have to be properly assessed. *Is this really a client I should trust?* Likewise, all data should be stored safely and all encryption keys held securely, and as we have to assume someone is listening, all sensitive data in transit within our system needs to be encrypted.

Interestingly, if you have properly implemented a zero-trust mindset, you can start to do things that seem pretty odd:

[With zero trust] you can actually make certain counter-intuitive access decisions and for example allow connections to internal services from the internet because you treat your "internal" network as equally trustworthy as the internet (i.e., not at all).

—Jan Schaumann[11]

Jan's argument here is that if you assume that nothing inside your network is to be trusted, and that trust has to be reestablished, you can be much more flexible about the environment in which your microservice lives—you aren't expecting the wider environment to be secure. But remember, zero trust isn't something you turn on with a switch. It's an underlying principle for how you decide to do things. It has to drive your decision making about how you build and evolve your system—it's going to be something that you have to constantly invest in to get the rewards.

It's a Spectrum

I don't mean to imply that you have a stark choice between implicit and zero trust. The extent to which you trust (or don't) other parties in your system could change based on the sensitivity of the information being accessed. You might decide, for example, to adopt a concept of zero trust for any microservices handling PII but be more relaxed in other areas. Again, the cost of any security implementation should be justified (and driven) by your threat model. Let your understanding of your threats and their associated impact drive your decision making around whether or not zero trust is worth it to you.

As an example, lets look at MedicalCo, a company I worked with that managed sensitive healthcare data pertaining to individuals. All information it held was classified based on a fairly sensible and straightforward approach:

Public

Data that could be freely shared with any external party. This information is effectively in the public domain.

Private

Information that should be available only to logged-in users. Access to this information could be further limited due to authorization restrictions. This might include things such as which insurance plan a customer was on.

Secret

Incredibly sensitive information about individuals that could be accessed by people other than the individual in question only in extremely specific situations. This includes information about an individual's health data.

Microservices were then categorized based on the most sensitive data they used and had to run in a matching environment (zone) with matching controls, as we see in Figure 11-5. A microservice would have to run in the zone matching the most sensitive data it made use of. For example, a microservice running in the public zone could use only public data. On the other hand, a microservice that used public and private data had to run in the private zone, and a microservice that accessed secret information always had to run in the secret zone.

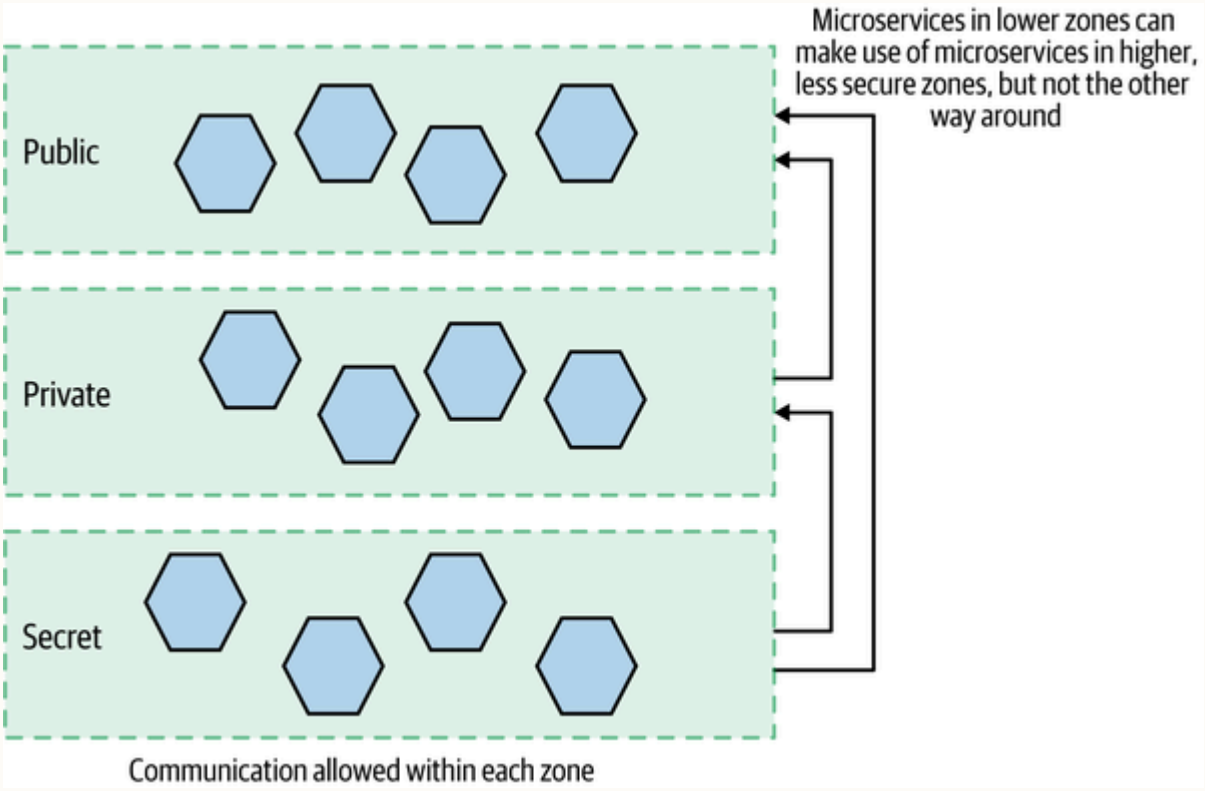


Figure 11-5. Deploying microservices into different zones based on the sensitivity of the data they handle

Microservices within each zone could communicate with each other but were unable to directly reach across to access data or functionality in the lower, more secure zones. Microservices in the more secure zones could reach up to access functionality running in the less secure zones, though.

Here, MedicalCo has given itself the flexibility to vary its approach in each zone. The less secure public zone can operate in something closer to an implicit trust environment, whereas the secret zone assumes zero trust. Arguably, if MedicalCo were to adopt a zero-trust approach across its entire system, having microservices deployed into separate zones wouldn't be required, as all inter-microservice calls would require additional authentication and authorization. That said, thinking of defense in depth once again, I can't help thinking I'd still consider this zoned approach given the sensitivity of the data!

Securing Data

As we break our monolithic software apart into microservices, our data moves around our systems more than before. It doesn't just flow over networks; it also sits on disk. Having more valuable data spread around more places can be a nightmare when it comes to securing our application, if we aren't careful. Let's look in more detail at how we can protect our data as it moves over networks, and as it sits at rest.

Data in Transit

The nature of the protections you have will depend largely on the nature of the communication protocols you have picked. If you are using HTTP, for example, it would be natural to look at using HTTP with Transport Layer Security (TLS), a topic we'll expand on more in the next section; but if you're using alternative protocols such as communication via a message broker, you may have to look at that particular technology's support for protecting data in transit. Rather than look at the details of a large range of technology in this space, I think instead it is important to consider more generically the four main areas of interest when it comes to securing data in transit, and to look at how these concerns could be addressed with HTTP as an example. Hopefully, it shouldn't be too difficult for you to map these ideas to whatever communication protocol you are picking.

In Figure 11-6 we can see the four key concerns of data in transit.

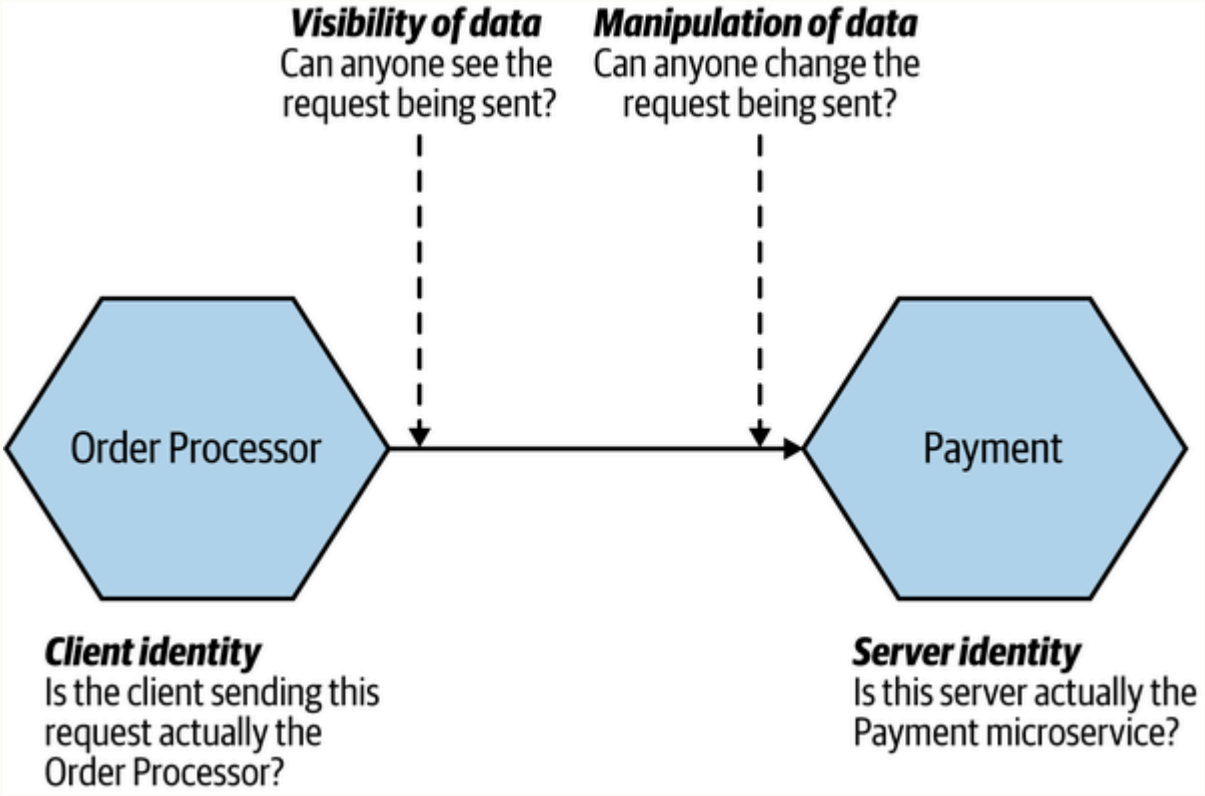


Figure 11-6. The four main concerns when it comes to data in transit

Let's look at each concern in a bit more detail.

Server identity

One of the simplest things to check is that the server you are talking to is exactly who it claims to be. This is important because a malicious party could theoretically impersonate an endpoint—and vacuum up any useful data that you send it. Validating server identity has long been a concern on the public internet, leading to the push for more widespread use of HTTPS—and to an extent, we are able to benefit from the work done to help secure the public internet when it comes to managing internal HTTP endpoints.

When people talk about "HTTPS," they are typically referring to using HTTP with TLS.[1][2] With most communication over the public internet, due to the various potential attack vectors out there (unsecured WiFi, DNS poisoning, and the like), it is vital to ensure that when we go to a website it really is the website it claims to be. With HTTPS, our browser can look at the certificate for that website and make sure it is valid. It is a highly sensible safety mechanism—"HTTPS Everywhere" has become a rallying cry for the public internet, and with good reason.

It's worth noting that some communication protocols that use HTTP under the hood can take advantage of HTTPS—so we can easily run SOAP or gRPC over HTTPS without issue. HTTPS also gives us additional protections over and above simply confirming that we are talking to whom we expect. We'll come to that shortly.

Client identity

When we refer to client identity in this context, we are referring to the microservice making a call—so we are trying to confirm and authenticate the identity of the upstream microservice. We will be looking at how we authenticate humans (users!) a bit later on.

We can verify the identity of a client in a number of ways. We could ask that the client send us some information in the request telling us who they are. An example might be to use some sort of shared secret or a client-side certificate to sign the request. When the server has to verify the client identity, we want this to be as efficient as possible—I've seen some solutions (including those pushed by API gateway vendors) that involve the server having to make calls to central services to check client identity, which is pretty nuts when you consider the latency implications.

I struggle to think of a situation in which I would verify client identity without also verifying server identity—to verify both, you would typically end up implementing some form of *mutual authentication*. With mutual authentication, both parties authenticate

each other. So in Figure 11-6, the `Order Processor` authenticates the `Payment` microservice, and the `Payment` microservice authenticates the `Order Processor`.

We can do this through the use of *mutual TLS*, in which case both the client and server make use of certificates. On the public internet, verifying the identity of a client device is typically less important than verifying the identity of the human using that device. As such, mutual TLS is rarely used. In our microservice architecture, though, especially where we might be operating in a zero-trust environment, this is much more common.

The challenge with implementing schemes like mutual TLS has historically been tooling. Nowadays this is less of an issue. Tools like Vault can make distributing certificates much easier, and wanting to simplify the use of mutual TLS is one of the main reasons for people to implement service meshes, which we explored in "Service Meshes and API Gateways".

Visibility of data

When we send data from one microservice to another, can someone view the data? For some information, such as the price of Peter Andre albums, we might not care much since the data is already in the public domain. On the other hand, some data might include PII, which we need to make sure is protected.

When you use either plain old HTTPS or mutual TLS, data won't be visible to intermediate parties—this is because TLS encrypts the data being sent. This can be problematic if you explicitly want the data sent in the open—for example, reverse proxies like Squid or Varnish are able to cache HTTP responses, but that isn't possible with HTTPS.

Manipulation of data

We could imagine a number of situations in which manipulating data being sent could be bad—changing the amount of money being sent, for example. So in Figure 11-6, we need to make sure the potential attacker is unable to change the request being sent to `Payment` from the `Order Processor`.

Typically, the types of protections that make data invisible will also ensure that the data can't be manipulated (HTTPS does that, for instance). However, we could decide to send data in the open but still want to ensure it cannot be manipulated. For HTTP, one such approach is to use a *hash-based message authentication code* (HMAC) to sign the data being sent. With HMAC, a hash is generated and sent with the data, and the receiver can check the hash against the data to confirm that the data hasn't been changed.

Data at Rest

Data lying about is a liability, especially if it is sensitive. Hopefully we've done everything we can to ensure attackers cannot breach our network, and also that they cannot breach our applications or operating systems to get access to the underlying data. However, we need to be prepared in case they do—defense in depth is key.

Many of the high-profile security breaches we hear of involve data at rest being acquired by an attacker, and that data being readable by the attacker. This happens either because the data was stored in an unencrypted form or because the mechanism used to protect the data had a fundamental flaw.

The mechanisms by which data at rest can be protected are many and varied, but there are some general things to bear in mind.

Go with the well known

In some cases, you can offload the job of encrypting data to existing software—for example, by making use of your database's built-in support for encryption. However, if you find a need to encrypt and decrypt data in your own system, make sure you're going with well-known and tested implementations. The easiest way you can mess up data encryption is to try to implement your own encryption algorithms, or even try to implement someone else's. Whatever programming language you use, you'll have access to reviewed, regularly patched implementations of well-regarded encryption algorithms. Use those! And subscribe to the mailing lists/advisory lists for the technology you choose to make sure you are aware of vulnerabilities as they are found, so you can keep them patched and up to date.

For securing passwords, you should absolutely be using a technique called *salted password hashing*. This ensures that passwords are never held in plain text, and that even if an attacker brute-forces one hashed password they cannot then automatically read other passwords.[13]

Badly implemented encryption could be worse than having none, as the false sense of security (pardon the pun) can lead you to take your eye off the ball.

Pick your targets

Assuming everything should be encrypted can simplify things somewhat. There is no guesswork about what should or should not be protected. However, you'll still need to

think about what data can be put into logfiles to help with problem identification, and the computational overhead of encrypting everything can become pretty onerous and require more powerful hardware as a result. This is even more challenging when you're applying database migrations as part of refactoring schemas. Depending on the changes that are being made, the data may need to be decrypted, migrated, and reencrypted.

By subdividing your system into more fine-grained services, you might identify an entire data store that can be encrypted wholesale, but that is unlikely. Limiting this encryption to a known set of tables is a sensible approach.

Be frugal

As disk space becomes cheaper and the capabilities of databases improve, the ease with which bulk amounts of information can be captured and stored is improving rapidly. This data is valuable—not only to businesses themselves, which increasingly see data as a valuable asset, but equally to the users who value their own privacy. The data that pertains to an individual or that could be used to derive information about an individual must be the data that we are most careful with.

However, what if we made our lives a bit easier? Why not scrub as much information as possible that can be personally identifiable, and do it as soon as possible? When logging a request from a user, do we need to store the entire IP address forever, or could we replace the last few digits with *x*? Do we need to store someone's name, age, gender, and date of birth in order to provide them with product offers, or is their age range and postal code enough information?

The advantages to being frugal with data collection are manifold. First, if you don't store it, no one can steal it. Second, if you don't store it, no one (e.g., a governmental agency) can ask for it either!

The German phrase *Datensparsamkeit* represents this concept. Originating from German privacy legislation, it encapsulates the concept of storing only as much information as is *absolutely required* to fulfill business operations or satisfy local laws.

This is obviously in direct tension with the move toward storing more and more information, but realizing that this tension even exists is a good start!

It's all about the keys

Most forms of encryption involves making use of some key in conjunction with a suitable algorithm to create encrypted data. To decrypt the data so it can be read, authorized parties will need access to a key—either the same key or a different key (in the case of public-key encryption). So where are your keys stored? Now, if I am encrypting my data because I am worried about someone stealing my whole database, and I store the key I use in the same database, then I haven't really achieved much! Therefore, we need to store the keys somewhere else. But where?

One solution is to use a separate security appliance to encrypt and decrypt data. Another is to use a separate key vault that your service can access when it needs a key. The life-cycle management of the keys (and access to change them) can be a vital operation, and these systems can handle this for you. This is where HashiCorp's Vault can also come in very handy.

Some databases even include built-in support for encryption, such as SQL Server's Transparent Data Encryption, which aims to handle this in a transparent fashion. Even if your database of choice does include such support, research how the keys are handled and understand if the threat you are protecting against is actually being mitigated.

Again, this stuff is complex. Avoid implementing your own encryption, and do some good research!

TIP

Encrypt data when you first see it. Only decrypt on demand, and ensure that data is never stored anywhere.

Encrypt backups

Backups are good. We want to back up our important data. And it may seem like an obvious point, but if the data is sensitive enough that we want it to be encrypted in our running production system, then we will probably also want to make sure that any backups of the same data are also encrypted!

Authentication and Authorization

Authentication and authorization are core concepts when it comes to people and things that interact with our system. In the context of security, *authentication* is the process by which we confirm that a party is who they say they are. We typically authenticate a human user by having them type in their username and password. We assume that only the actual user has access to this information, and therefore the person entering this information must be them. Other, more complex systems exist as well, of course. Our phones now let us use our fingerprint or face to confirm that we are who we say we are. Generally, when we're talking abstractly about who or what is being authenticated, we refer to that party as the *principal*.

Authorization is the mechanism by which we map from a principal to the action we are allowing them to do. Often, when a principal is authenticated, we will be given information about them that will help us decide what we should let them do. We might, for example, be told what department or office they work in—a piece of information that our system can use to decide what the principal can and cannot do.

Ease of use is important—we want to make it easy for our users to access our system. We don't want everyone to have to log in separately to access different microservices, using a different username and password for each one. So we also need to look at how we can implement single sign-on (SSO) in a microservices environment.

Service-to-Service Authentication

Earlier we discussed mutual TLS, which, aside from protecting data in transit, also allows us to implement a form of authentication. When a client talks to a server using mutual TLS, the server is able to authenticate the client, and the client is able to authenticate the server—this is a form of service-to-service authentication. Other authentication schemes can be used besides mutual TLS. A common example is the use of API keys, where the client needs to use the key to hash a request in such a way that the server is able to verify that the client used a valid key.

Human Authentication

We're accustomed to humans authenticating themselves with the familiar username and password combination. Increasingly, though, this is being used as part of a multifactor authentication approach, where a user may need more than one piece of knowledge (a *factor*) to authenticate themselves. Most commonly, this takes the form of multifactor

authentication (MFA),[14] where more than one factor is needed. MFA would most commonly involve the use of a normal username and password combo, in addition to providing at least one additional factor.

The different types of authentication factors have grown in recent years—from codes sent over SMS and magic links sent via email to dedicated mobile apps like Authy and USB and NFC hardware devices like the YubiKey. Biometric factors are more commonly used now as well, as users have more access to hardware that supports things like fingerprint or face recognition. While MFA has shown itself to be much more secure as a general approach, and many public services support it, it hasn't caught on as a mass market authentication scheme, although I do expect that to change. For managing authentication of key services that are vital to the running of your software or allow access to especially sensitive information (e.g., source code access), I'd consider the use of MFA to be a must.

Common Single Sign-On Implementations

A common approach to authentication is to use some sort of single sign-on (SSO) solution to ensure that a user has only to authenticate themselves only once per session, even if during that session they may end up interacting with multiple downstream services or applications. For example, when you log in with your Google account, you are logged in on Google Calendar, Gmail, and Google Docs, even though these are separate systems.

When a principal tries to access a resource (like a web-based interface), they are directed to authenticate with an *identity provider*. The identity provider may ask them to provide a username and password or might require the use of something more advanced like MFA. Once the identity provider is satisfied that the principal has been authenticated, it gives information to the *service provider*, allowing it to decide whether to grant them access to the resource.

This identity provider could be an externally hosted system or something inside your own organization. Google, for example, provides an OpenID Connect identity provider. For enterprises, though, it is common to have your own identity provider, which may be linked to your company's *directory service*. A directory service could be something like the Lightweight Directory Access Protocol (LDAP) or Active Directory. These systems allow you to store information about principals, such as what roles they play in the organization. Often the directory service and the identity provider are one and the same, while at other times they are separate but linked. Okta, for example, is a hosted SAML identity

provider that handles tasks like two-factor authentication but can link to your company's directory services as the source of truth.

So the identity provider gives the system information about who the principal is, but the system decides what that principal is allowed to do.

SAML is a SOAP-based standard and is known for being fairly complex to work with despite the libraries and tooling available to support it, and since the first edition of this book it has rapidly fallen out of favor.[15] OpenID Connect is a standard that has emerged as a specific implementation of OAuth 2.0, based on the way Google and others handle SSO. It uses simpler REST calls, and due in part to its relative simplicity and widespread support, it is the dominant mechanism for end-user SSO, and has gained significant inroads into enterprises.

Single Sign-On Gateway

We could decide to handle the redirection to, and handshaking with, the identity provider within each microservice, so that any unauthenticated request from an outside party is properly dealt with. Obviously, this could mean a lot of duplicated functionality across our microservices. A shared library could help, but we'd have to be careful to avoid the coupling that can come from shared code (see "DRY and the Perils of Code Reuse in a Microservice World" for more). A shared library also wouldn't help if we had microservices written in different technology stacks.

Rather than having each service manage handshaking with our identity provider, a more common approach is to use a gateway to act as a proxy, sitting between your services and the outside world (as shown in Figure 11-7). The idea is that we can centralize the behavior for redirecting the user and perform the handshake in only one place.

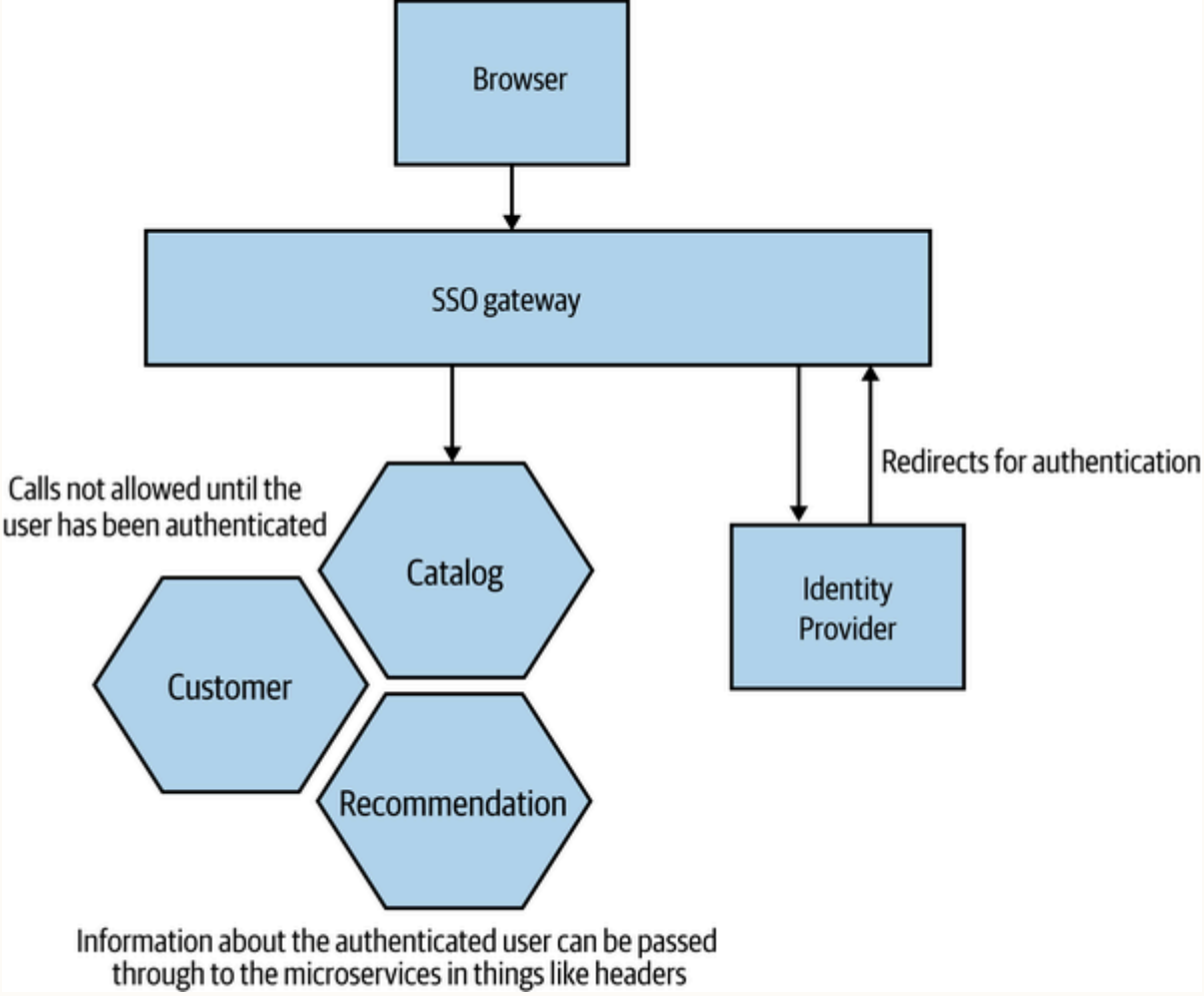


Figure 11-7. Using a gateway to handle SSO

However, we still need to solve the problem of how the downstream service receives information about principals, such as their username or what roles they play. If you're using HTTP, you could configure your gateway to populate headers with this information. Shibboleth is one tool that can do this for you, and I've seen it used with the Apache web server to handle integration with SAML-based identity providers, to great effect. An alternative, which we'll look at in more detail shortly, is to create a JSON Web Token (JWT) containing all the information about the principal; this has a number of benefits, including being something we can more easily pass from microservice to microservice.

Another consideration with using a single sign-on gateway is that if we have decided to offload responsibility for authentication to a gateway, it can be harder to reason about how a microservice behaves when looking at it in isolation. Remember in Chapter 9 when we explored some of the challenges of reproducing production-like environments? If you decide to use a gateway, make sure your developers can launch their services behind one without too much work.

One final problem with this approach is that it can lull you into a false sense of security. Again, I like to return to the idea of defense in depth—from network perimeter to subnet, firewall, machine, operating system, and the underlying hardware. You have the ability to implement security measures at all of these points. I have seen some people put all

their eggs in one basket, relying on the gateway to handle every step for them. And we all know what happens when we have a single point of failure...

Obviously you could use this gateway to do other things. For example, you could also decide to terminate HTTPS at this level, run intrusion detection, and so on. Do be careful, though. Gateway layers tend to take on more and more functionality, which itself can end up being a giant coupling point. And the more functionality something has, the greater the attack surface.

Fine-Grained Authorization

A gateway may be able to provide fairly effective coarse-grained authentication. For example, it could prevent any non-logged-in users from accessing the helpdesk application. Assuming our gateway can extract attributes about the principal as a result of the authentication, it may be able to make more nuanced decisions. For example, it is common to place people in groups or assign them to roles. We can use this information to understand what they can do. So for the helpdesk application, we might allow access only to principals with a specific role (e.g., `STAFF`). Beyond allowing (or disallowing) access to specific resources or endpoints, though, we need to leave the rest to the microservice itself; it will need to make further decisions about what operations to allow.

Back to our helpdesk application: do we allow any staff member to see any and all details? More likely, we'll have different roles at work. For example, a principal in the `CALL_CENTER` group might be allowed to view any piece of information about a customer except their payment details. This principal might also be able to issue refunds, but that amount might be capped. Someone who has the `CALL_CENTER_TEAM_LEADER` role, however, might be able to issue larger refunds.

These decisions need to be local to the microservice in question. I have seen people use the various attributes supplied by identity providers in horrible ways, using really fine-grained roles like `CALL_CENTER_50_DOLLAR_REFUND`, where they end up putting information specific to one piece of microservice functionality into their directory services. This is a nightmare to maintain and gives very little scope for our services to have their own independent life cycle, as suddenly a chunk of information about how a service behaves lives elsewhere, perhaps in a system managed by a different part of the organization.

Making sure the microservice has the information it needs to assess finer-grained authorization requests is worthy of further discussion—we'll revisit this when we look at JWTs in a moment.

Instead, favor coarse-grained roles modeled around how your organization works. Going all the way back to the early chapters, remember that we are building software to match how our organization works. So use your roles in this way too.

The Confused Deputy Problem

Having a principal authenticate with the system as a whole using something like an SSO gateway is simple enough, and this could be enough to control access to a given microservice. But what happens if that microservice then needs to make additional calls to complete an operation? This can leave us open to a type of vulnerability known as the confused deputy problem. This occurs when an upstream party tricks an intermediate party into doing things it shouldn't be doing. Let's look at a concrete example in Figure 11-8, which illustrates MusicCorp's online shopping site. Our browser-based JavaScript UI talks to the server-side `Web Shop` microservice, which is a type of backend for frontend. We'll explore this in more depth in "Pattern: Backend for Frontend (BFF)", but for the moment, think of it as a server-side component that performs call aggregation and filtering for a specific external interface (in our case, our browser-based JavaScript UI). Calls made between the browser and the `Web Shop` can be authenticated using OpenID Connect. So far, so good.

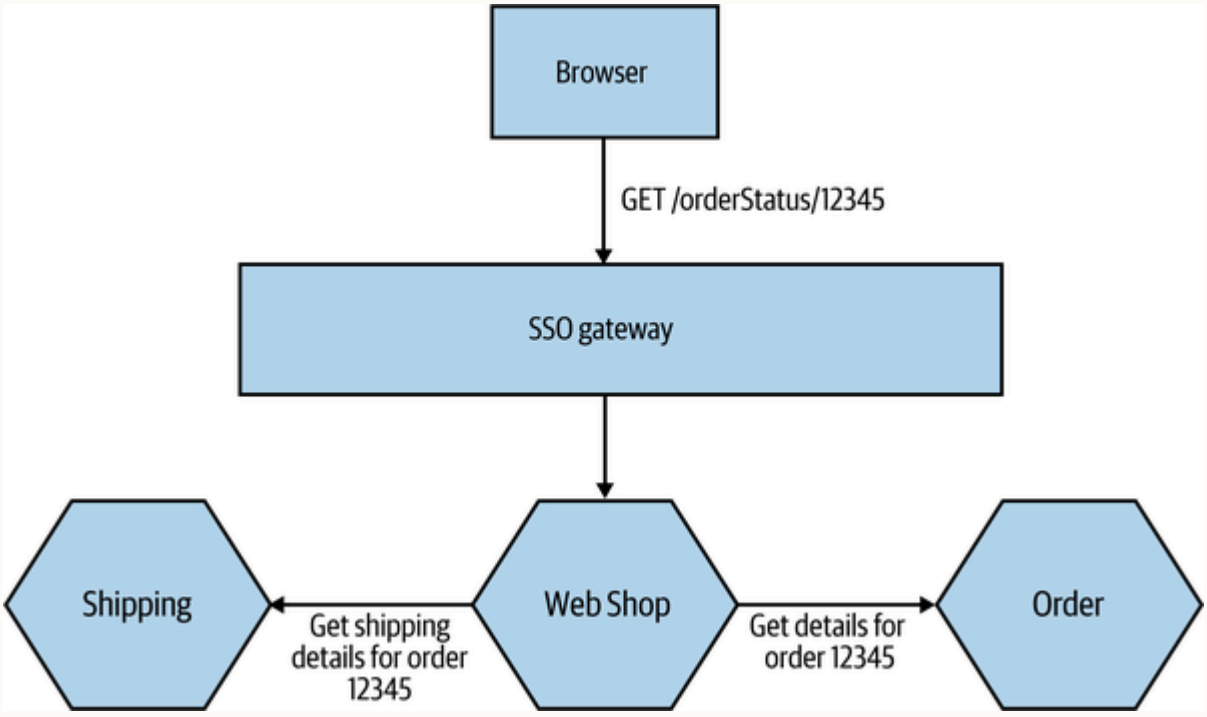


Figure 11-8. An example where a confused deputy could come into play

When a user is logged in, they can click on a link to view details of an order. To display the information, we need to pull back the original order from the `Order` service, but we also want to look up shipping information for the order. So when a logged-in customer clicks a link for `/orderStatus/12345`, this request is routed to `Web Shop`, which then

needs to make calls to the downstream `Order` and `Shipping` microservices asking for the details for order `12345`.

But should these downstream services accept the calls from the `Web Shop`? We could adopt a stance of implicit trust: because the call came from within our perimeter, it is OK. We could even use certificates or API keys to confirm that it really is the `Web Shop` asking for this information. But is this enough? For example, a customer who is logged into the online shopping system can see their personal account details. What if the customer could trick the web shop's UI into making a request for *someone else's* details, maybe by making a call with their own logged-in credentials?

In this example, what is to stop the customer from asking for orders that are not theirs? Once logged in, they could start sending requests for other orders that aren't theirs to see if they could get useful information. They could start guessing order IDs to see if they could extract other people's information. Fundamentally, what has happened here is that while we have *authenticated* the user in question, we aren't providing sufficient *authorization*. What we want is some part of our system to be able to judge that a request to see User A's details can be granted only if it's User A asking to see them. Where does this logic live, though?

Centralized, Upstream Authorization

One option for avoiding the confused deputy problem is to perform all required authorization as soon as the request is received in our system. In Figure 11-8, this would mean that we would aim to authorize the request either in the SSO gateway itself or in the `Web Shop`. The idea is that by the time the calls are sent to the `Order` or `Shipping` microservice, we assume that the requests are allowed.

This form of upstream authorization effectively implies that we are accepting some form of implicit trust (as opposed to zero trust)—the `Shipping` and `Order` microservices have to assume they are only being sent requests that they are allowed to fulfill. The other issue is that an upstream entity—for example, a gateway or something similar—needs to have knowledge of what functionality downstream microservices provide, and it needs to know how to limit access to that functionality.

Ideally, though, we want our microservices to be as self-contained as possible, to make it as easy as possible to make changes and roll out new functionality. We want our releases to be as simple as possible—we want independent deployability. If the act of deployment now involves both deploying a new microservice and applying some authoriza-

tion-related configuration to an upstream gateway, then that doesn't seem terribly "independent" to me.

Therefore, we'd want to push the decision about whether or not the call should be authorized into the same microservice where the functionality being requested lives. This makes the microservice more self-contained and also gives us the option to implement zero trust if we want.

Decentralizing Authorization

Given the challenges of centralized authorization in a microservices environment, we'd like to push this logic to the downstream microservice. The `Order` microservice is where the functionality for accessing order details lives, so it would make logical sense for that service to decide if the call is valid. In this specific case, though, the `Order` microservice needs information about what human is making the request. So how do we get that information to the `Order` microservice?

At the simplest level, we could just require that the identifier for the person making the request be sent to the `Order` microservice. If using HTTP, for example, we could just stick the username in a header. But in such a case, what's to stop a malicious party from inserting just any old name into the request to get the information they need? Ideally, we want a way to make sure that the request is *really* being made on behalf of an authenticated user and that we can pass along additional information about that user—for example, the groups that user might fall in.

Historically, there are a variety of different ways that this has been handled (including techniques like nested SAML assertions, which, yes, are as painful as they sound), but recently the most common solution to this particular problem has been to use JSON Web Tokens.

JSON Web Tokens

JWTs allow you to store multiple claims about an individual into a string that can be passed around. This token can be signed to ensure that the structure of the token hasn't been manipulated, and it can also optionally be encrypted to provide cryptographic guarantees around who can read the data. Although JWTs can be used for generic exchanges of information where it is important to ensure that the data hasn't been tampered with, they are most commonly used to help transmit information to aid authorization.

Once signed, the JWTs can easily be passed via a variety of protocols, and the tokens can optionally be configured to expire after a certain period of time. They are widely supported, with a number of identity providers that support generating JWTs, and a large number of libraries for using JWTs inside your own code.

Format

The main payload of a JWT is a JSON structure, which broadly speaking can contain anything you want. We can see an example token in Example 11-1. The JWT standard does describe some specifically named fields ("public claims") that you should use if they relate to you. For example, `exp` defines the expiration date of a token. If you use these public claim fields correctly, there is a good chance that the libraries you use will be able to make use of them appropriately—rejecting a token, for example, if the `exp` field states that the token has already expired.[16] Even if you won't use all of these public claims, it's worth being aware of what they are to ensure you don't end up using them for your own application-specific uses, as this could cause some odd behavior in supporting libraries.

Example 11-1. An example of the JSON payload of a JWT

```
{
  "sub": "123",
  "name": "Sam Newman",
  "exp": 1606741736,
  "groups": "admin, beta"
}
```

In Example 11-2, we see the token from Example 11-1 encoded. This token is really just a single string, but it is split into three parts delineated with a "."—the header, payload, and signature.

Example 11-2. The result of encoding a JWT payload

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9. ❶
eyJzdWIiOiIxMjMiLCJuYW1lIjoiU2FtIE5ld21hbiIsImV4cCI6MTYwNjc0MTczNiwiZ3J... . ❷
Z9HMHODGs6OIOP5bVVSFixeDxJjGovQEtlNUi_iE_0 ❸
```

❶ The header

❷ The payload (truncated)

❸ The signature

For the benefit of the example here, I have split the line at each part, but in reality this would be a single string with no line breaks. The header contains information about the signing algorithm being used. This allows the program decoding the token to support different signing schemes. The payload is where we store information about the claims that the token is making—this is just the result of encoding the JSON structure in Example 11-1. The signature is used to ensure that the payload hasn't been manipulated, and it can also be used to ensure that the token was generated by whom you think it was (assuming the token is signed with a private key).

As a simple string, this token can be easily passed along via different communication protocols—as a header in HTTP (in the `Authorization` header), for example, or perhaps as a piece of metadata in a message. This encoded string can of course be sent over encrypted transport protocol—for example, TLS over HTTP—in which case the token wouldn't be visible to people observing the communication.

Using tokens

Let's take a look at a common way to use JWT tokens in a microservice architecture. In Figure 11-9, our customer logs in as normal, and once authenticated we generate some sort of token to represent their logged-in session (likely an OAuth token), which is stored on the client device. Subsequent requests from that client device hit our gateway, which generates a JWT token that will be valid for the duration of that request. It is this JWT token that is then passed to the downstream microservices. They are able to validate the token and extract claims from the payload to determine what sort of authorization is appropriate.

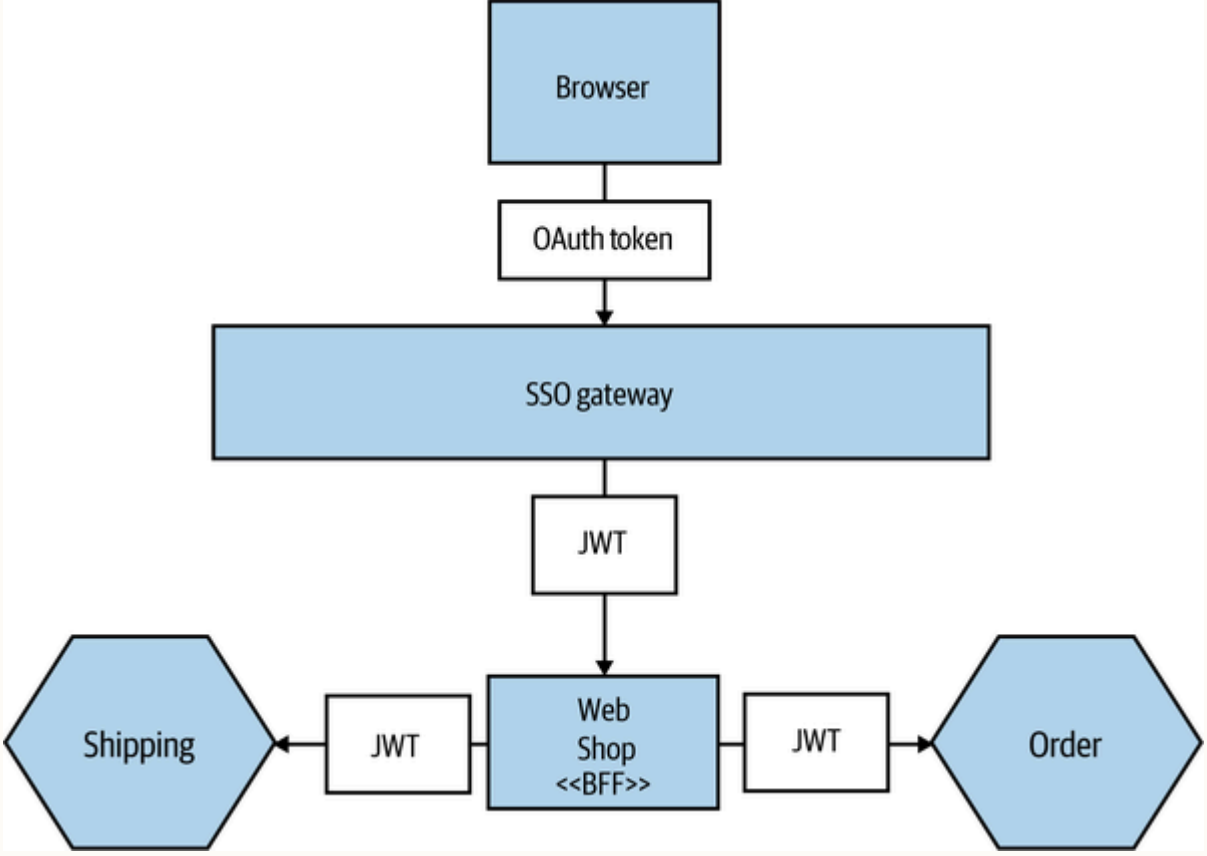


Figure 11-9. A JWT token is generated for a specific request and passed to downstream microservices

A variation to this approach is to generate a JWT token when the user initially authenticates themselves with the system and then have that JWT token stored on the client device. It's worth considering, though, that such a token will have to be valid for the duration of the logged-in session; as we've already discussed, we'd like to limit the period of validity for system-generated credentials to reduce the chances of them being misused, and to reduce the impact if we need to change the keys that are used for generating the encoded token. Generating a JWT token on a per-request basis seems to be the most common solution to this problem, as we show in Figure 11-9. Having some sort of token exchange done in the gateway can also make it much easier to adopt the use of JWT tokens without having to change any part of the authentication flow that involves communication with the client device—if you already have a working SSO solution in place, hiding the fact that JWT tokens are even used from the main user authentication flow will make such a change less disruptive.

So with suitable JWT token generation, our downstream microservices are able to get all the information they need to confirm the identity of the user making the request, as well as additional information such as the groups or roles that user is in. The validity of this token can also be checked by the microservice simply by checking the signature of the JWT token as well. Compared to the previous solutions in this space (such as nested SAML assertions), JWT tokens have made the process of decentralizing authorization in a microservice architecture much simpler.

Challenges

There are a few issues around JWT tokens that are worth bearing in mind. The first is the issue of keys. In the case of signed JWT tokens, to verify a signature, the receiver of a JWT token is going to need some information that will need to be communicated out of band—normally a public key. All the issues of key management apply in this case. How does the microservice get the public key? What happens if the public key needs to change? Vault is an example of a tool that could be used by a microservice to retrieve (and handle rotation of) public keys, and it is already designed to work in a highly distributed environment. You could of course just hardcode a public key in a configuration file for the receiving microservice, but you would then have the issue of dealing with the public key changing.

Secondly, getting the expiration right for a token can be tricky if long processing times are involved. Consider a situation in which a customer has placed an order. This kicks off a set of asynchronous processes that could take hours if not days to complete, without any subsequent involvement from the customer (taking payment, sending notification emails, getting the item packaged and shipped, etc.). Do you therefore have to generate a token with a matching period of validity? The question here is, at what point is having a longer-lived token more problematic than having no token? I've spoken to a few teams that have dealt with this issue. Some have generated a special longer-lived token that is scoped to work in only this specific context; others have just stopped using the token at a certain point in the flow. I've not yet looked at enough examples of this problem to determine the right solution here, but it is an issue to be aware of.

Finally, in some situations you can end up needing so much information in the JWT token that the token size itself becomes a problem. Although this situation is rare, it does happen. Several years ago I was chatting to one team about using a token to manage authorization for a particular aspect of its system that handled rights management for music. The logic around this was incredibly complex—my client worked out that for any given track it could potentially need up to 10,000 entries in a token to deal with the different scenarios. We realized, though, that at least in that domain, it was only one particular use case that needed this large amount of information, whereas the bulk of the system could make do with a simple token with fewer fields. In such a situation, it made sense to deal with the more complex rights management authorization process in a different way—essentially using the JWT token for the initial "simple" authorization, and then doing a subsequent lookup on a data store to fetch the additional fields as required. This meant the bulk of the system could just work off the tokens.

Summary

As I hope I've articulated in this chapter, building a secure system isn't about doing one thing. It necessitates a holistic view of your system, using some sort of threat modeling exercise, to understand what type of security controls need to be put in place.

When thinking of these controls, a mix is essential to building a secure system. Defense in depth doesn't just mean having multiple protections in place; it also means you have a multifaceted approach to building a more secure system.

We also return again to a core theme of the book: having a system decomposed into finer-grained services gives us many more options for solving problems. Not only can having microservices potentially reduce the impact of any given breach, but it also allows us to consider trade-offs between the overhead of more complex and secure approaches, where data is sensitive, and a more lightweight approach when the risks are lower.

For a broader look at application security in general, I recommend *Agile Application Security* by Laura Bell et al.[17]

Next, we'll look at how we can make our systems more reliable, as we turn to the topic of resilience.

1 Please, let's not make everything about Brexit.

2 Try as I might, I can't find the original source for this categorization scheme.

3 I recommend Jonny's *Understanding Design Thinking, Lean, and Agile* (O'Reilly) for more of his insights.

4 It was more of a passive-aggressive argument, often without the "passive" bit.

5 Adam Shostack, *Threat Modeling: Designing for Security* (Indianapolis: Wiley, 2014).

6 Troy Hunt, "Passwords Evolved: Authentication Guidance for the Modern Era," July 26, 2017, *https://oreil.ly/T7PYM*.

7 Neil McAllister, "Code Spaces Goes Titsup FOREVER After Attacker NUKES Its Amazon-Hosted Data," *The Register*, June 18, 2014, *https://oreil.ly/mw7PC*.

8 Some people are concerned about the fact that secrets are stored in plain text. Whether or not this is an issue for you depends largely on your threat model. For the secrets to be read, an attacker would have to have direct access to the core systems running your cluster, at which point it's arguable that your cluster is already hopelessly compromised.

9 I definitely came up with this term, but I also think it highly probable that I'm not the only one.

10 Just as Niels Bohr argued that Schrödinger's cat was both alive and dead until you actually opened the box to check.

11 Jan Schaumann (@jschauma), Twitter, November 5, 2020, 4:22 p.m., *https://oreil.ly/QaCm2*.

12 The "S" in "HTTPS" used to relate to the older Secure Socket Layer (SSL), which has been replaced with TLS for a number of reasons. Confusingly, the term *SSL* still lingers even when TLS is actually being used. The OpenSSL library, for example, is actually widely used to implement TLS, and when you are issued an SSL certificate, it will actually be for TLS. We don't make things easy for ourselves, do we?

13 We don't encrypt passwords at rest, as encryption means that anyone with the right key can read the password back.

14 Previously we would talk about two-factor authentication (2FA). MFA is the same concept but introduces the idea that we often now allow our users to provide an additional factor from a variety of devices, such as a secure token, a mobile authentication app, or perhaps biometrics. You can consider 2FA to be a subset of MFA.

15 I can't take credit for that!

16 The JWT site has an excellent overview of which libraries support which public claims—it's a great resource in general for all things JWT.

17 Laura Bell et al., *Agile Application Security* (Sebastopol: O'Reilly, 2017).

Previous chapter

< 10. From Monitoring to Observability

Next chapter

12. Resiliency >